



Members: Beau Brechtel, Kaidon Eccles, Nicholas Schroeder, Sharmin Zaman, Nathan Evans

P1

Working Title: Hamster Homebound

Hamster Homebound is a 2D platformer inspired by classics like Super Mario Bros. and the Red Ball series. The story follows a small but determined hamster who becomes separated from his family during a sudden storm. Swept far from home, he must journey across a series of whimsical but dangerous environments to reunite with them. Along the way, he encounters moving platforms, environmental hazards, enemies such as snakes, and a variety of power-ups that either help or hinder his progress. Each level brings him one step closer to finding his family, who have been captured and locked away in a cage. We think this will be an exciting game because each level introduces new mechanics, enemies, and puzzles through dynamic platforms, which have proven to be entertaining for players of similar games.

Characters: Hamster, Snakes, Hamster's family

Objects:

- Pushable boxes (used for platforming puzzles or reaching higher areas)
- Stars (collectibles that determine score out of 3 stars at the end of the level)
- Pears (can restore 1 life)
- Keys (required to unlock gates or the final cage holding the hamster's family)

Objectives:

- Primary Objective: Reach the picket sign at the end of each level to progress.
- Secondary Objective: Collect keys scattered throughout levels.
- Long-Term Objective: Reunite the hamster with his family by completing all levels and unlocking their cage.

Procedures:

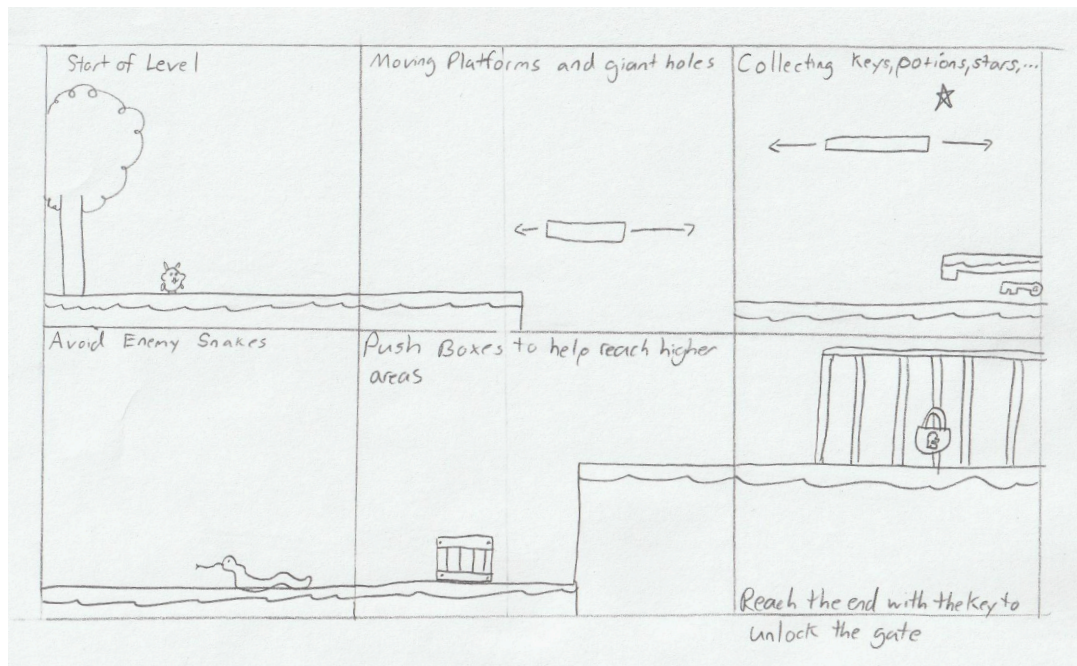
- Movement: Run, jump, double-jump (if unlocked), push objects.
- Combat/Evasion: Avoid or jump over enemies; possibly stomp on them if the team chooses.
- Interaction: Pick up keys, use pears.
- Level Progression: Reach the end-of-level marker to advance to the next stage.

Rules:

1. The hamster has three lives, and if he loses them all, he dies, game over

2. The hamster loses a life when touching an enemy or falling into hazards (water, spikes, pits).
3. Keys must be collected to unlock certain doors or the final cage.
4. A pear can restore a life
5. Some boxes can be pushed; others are static and used as platforms.
6. Levels must be completed in order.

Outcome: Hamster gets through all of the levels, collects every key, and uses them to unlock the cage his family is in to set them free



Main Character: Hamster (Player Character)

Name: *(working name)* Hammy

Role in the Game:

The hamster is the main playable character controlled by the player. His goal is to travel through each level, avoid hazards, collect keys and resources, and eventually rescue his family.

Visual Design

- Small, round body to emphasize cute and approachable style.
- Soft brown or tan fur with a lighter belly.
- Large expressive eyes to show emotion (determined, scared, happy).

- Tiny backpack or pouch (optional) to visually explain carrying keys or items.

Simple, readable silhouette so the character is easy to recognize during movement.

Movement & Animation

- Idle animation: Slight breathing or ear twitch to make the character feel alive.
- Running: Quick leg movement with dust trail for feedback.
- Jumping: Squash-and-stretch animation to make jumps feel responsive.
- Double-jump (unlockable): Small spin or flip animation.
- Damage animation: Brief blinking or knockback effect.

Victory animation: Cheer or happy jump at the end of a level.

Abilities

- Run left and right.
- Jump across platforms.
- Push movable boxes.
- Collect keys, stars, and potions.
- Unlock gates/cages using collected keys
- Optional unlockable ability: Double Jump for more advanced platforming

Gameplay Strengths

- Small size allows navigation through tight spaces.
- Agile movement suited for platform challenges.
- Can use objects in the environment to solve puzzles.

Weaknesses

- Limited to three lives.
- Vulnerable to enemies and environmental hazards.

Character Motivation

The hamster's primary motivation is to reunite with his captured family. His animation style, facial expressions, and progression through levels reinforce the emotional goal of returning home.

[20 pts]E. Paper Prototype Video

P2

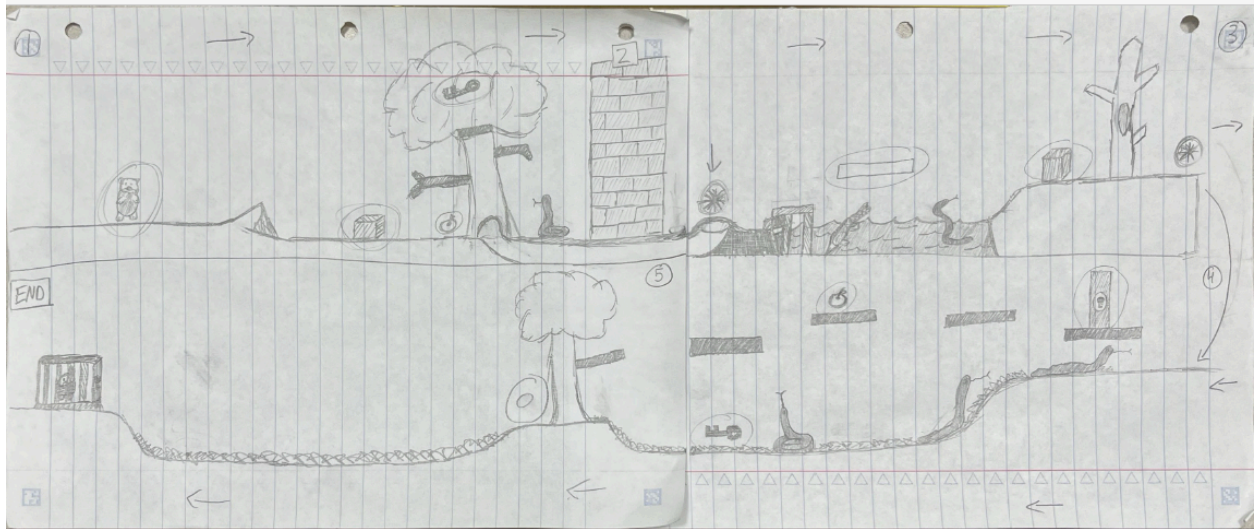
Hi Fidelity Prototype



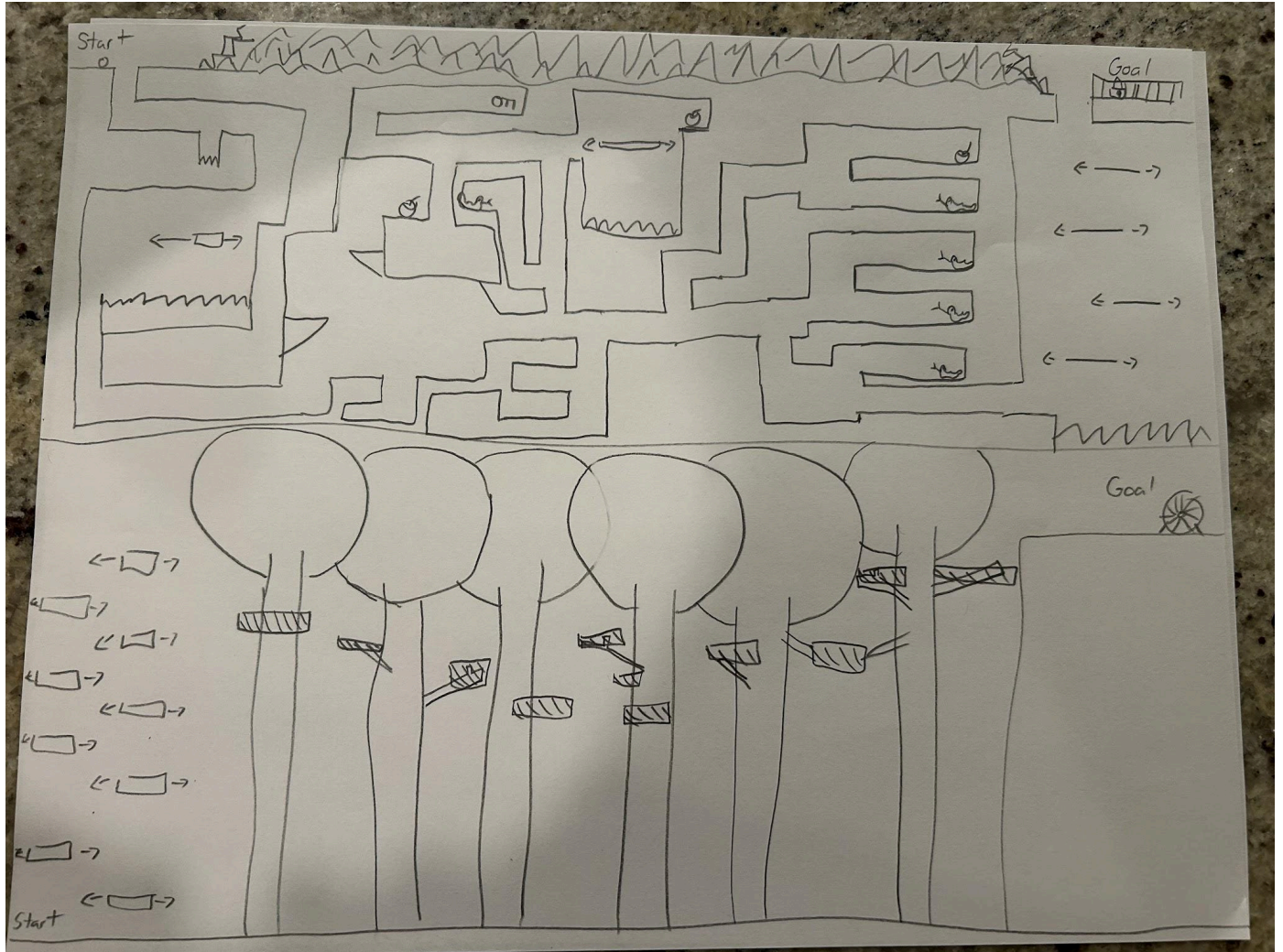


Level Designs:

From P1



Underground Level and Treetop Level



Potential Assets:
Pixel Adventure 1

By Pixel Frog

P3

P3.Progress Report1

Members and Roles:

Sharmin: setting up project assets, camera, player movement (Following Udemty tutorial sections 1-4, videos 1-24: Introduction, Getting Started, Player, Camera, Making a Level)

Beau: Tilemap design for additional levels, health system, checkpoints & respawning (Tutorial sections 5-7, videos 26-42: Making a Level, Health System, Checkpoints & Respawnng)

Nick: Character animation, additional character design, sound implementation (Tutorial sections 8-10, videos 43-61: Pickups, Making Enemies, Audio)

Kaidon: Menus, level transitions, level select system (Tutorial sections 11-13, videos 62-88: Menus, Levels, Level Select)

Sharmin & Nathan: Advanced mechanics, boss battle, finalizing, extras (Tutorial sections 14-17, videos 89-115: More Level Mechanics, Boss Battle, Finalising, Extras)

Member contributions:

So far, we have developed the initial concept for our game's core mechanics through the physical prototype and translated those ideas into a functional early build inside Unity. This early build includes the foundational movement system, adjusted parallax camera behavior, and level layout that reflect the structure and flow of the physical prototype.

Game Plan

Our game, Hamster Homebound, is a 2D pixel-art platformer centered around Milton, a small hamster who escapes his cage and must navigate a series of environments to return home. Milton's journey takes him through increasingly complex levels that introduce new mechanics, enemies, and environmental challenges. The game is structured into layers that reflect the development process. Layer 1 focuses on core movement, jumping, collisions, and camera behavior. Layer 2 introduces tilemap-based level design, parallax backgrounds, and basic hazards. Layers 3 and 4 will expand the game with enemies, pickups, health systems, checkpoints, menus, level transitions, and a final boss encounter. At this stage, the team is completing Layers 1 and 2.

Progress so far (with screenshots)

Significant progress has been made on the foundational systems of the game. The project was created using Unity's Universal 2D template, and all necessary assets were imported and organized. Custom pixel art was placed under a dedicated Art folder, and sprite settings were configured to ensure crisp visuals, including adjusting Pixels Per Unit, filter modes, and compression settings.

We're following the Udemy tutorial: [Learn to Code By Making a 2D Platformer in Unity and C#](#) as a guide and writing down notes from the videos in a shared Google Doc, and we're using GitHub as our repo and Git Bash as our terminal for collaboration.

Assets > 2D Platformer Assets > Graphics > Player > player-idle:

In Inspector, change Pixels Per Unit to 16, change Filter Mode to Point (no filter), change Compression to None, change Sprite Mode to Multiple, scroll down, and click Apply button

Still in Inspector for player-idle, Go into Open Sprite Editor click the Slice dropdown, change Type to Grid by Cell Size, change Pixel Size to 32 by 32, change the x value of Padding to 1, hit the Apply button and close out.

Click the arrow next to player-idle and drag the first hamster image onto the scene. This becomes "player-idle_0" in Hierarchy. Delete the other one, "player-idle"

Fig 1. Process documented in video notes

The player character, Milton, has been fully implemented with a Rigidbody2D, CapsuleCollider2D, and a Physics Material to refine movement. Core movement and jumping mechanics are functional, including a double jump system. Ground detection was implemented using an OverlapCircle and a LayerMask, and several physics-related issues—such as infinite jumping and collider misalignment—were resolved through debugging.

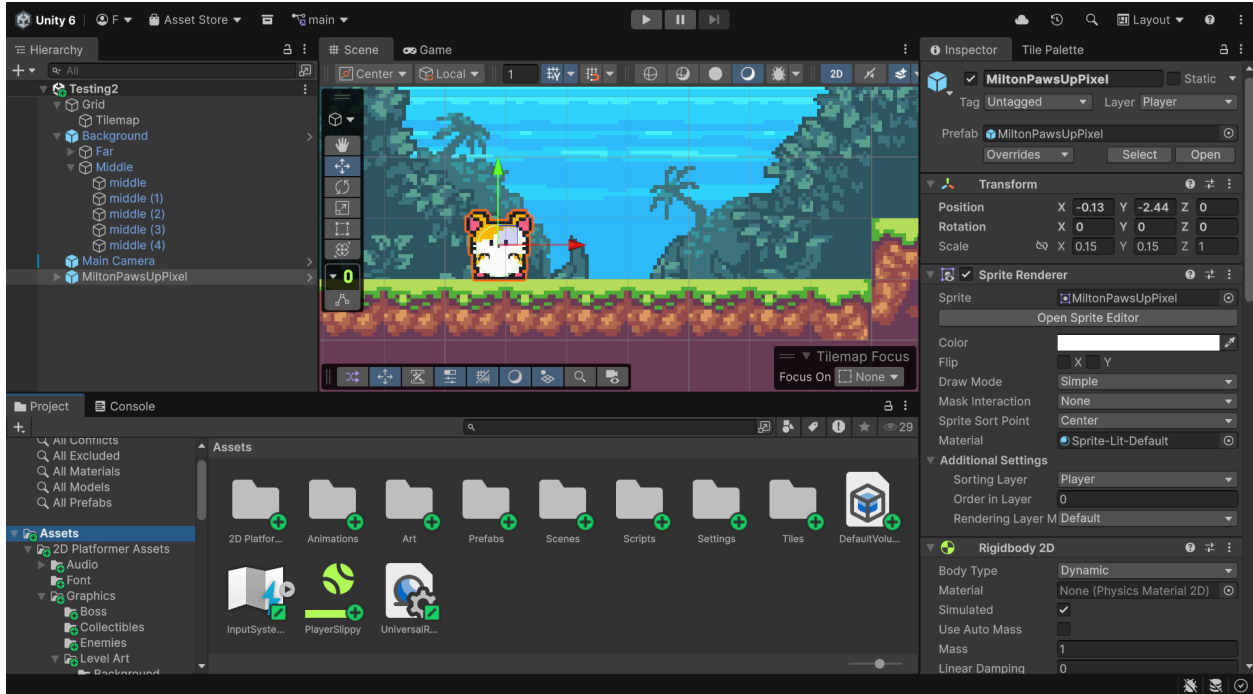


Fig. 2: Player character settings

The camera system is also complete, featuring smooth horizontal and vertical following, parallax scrolling for background layers, and vertical clamping to maintain framing. A major camera rendering issue, where the Game View displayed only a blue screen, was diagnosed and fixed by correcting the camera's Z-position.

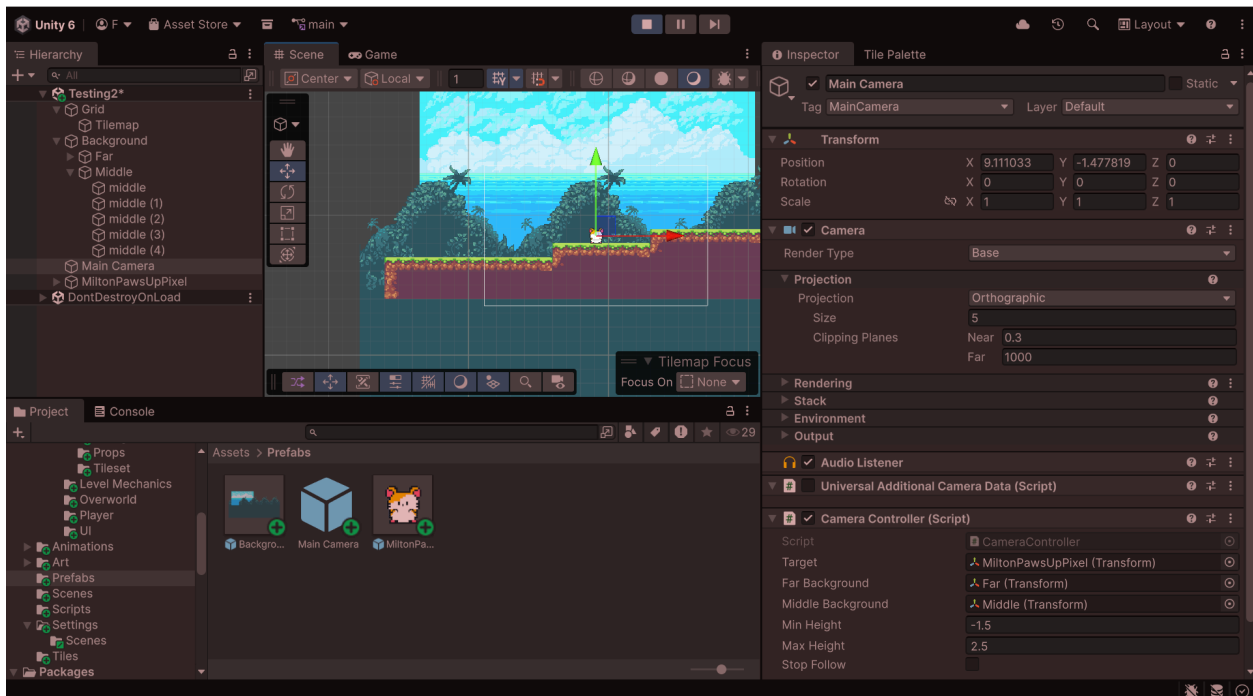


Fig. 3: Camera settings

Level design has begun using Unity's Tilemap system. A Tile Palette was created, and the first level, Testing2, was drawn using tiles from the imported asset pack. The Tilemap Collider 2D and Composite Collider 2D were added and configured to ensure that the ground behaves according to physics principles.

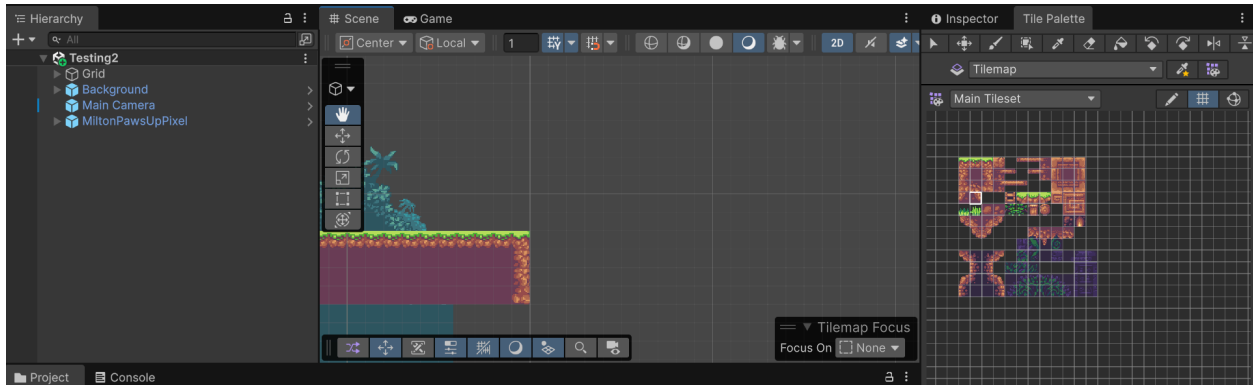


Fig. 4: Testing 2 tilemap creation

Sorting layers were established for backgrounds, world tiles, and the player. Prefabs were created for the player, camera, and background, enabling consistent behavior across multiple scenes. These prefabs were successfully imported into a second scene, and all necessary references were re-linked.

At this stage, Layer 1 is fully complete, and Layer 2 is well underway.

Player Controls/Actions

Milton can move left and right using the keyboard, jump using the spacebar, and perform a double jump while airborne. The player collides naturally with ground tiles and platforms, and the camera follows Milton smoothly across the level, incorporating parallax movement to create depth. These actions establish the basic feel of the game and will support more advanced mechanics in later layers.

Layer Details

Layer 1, which includes movement, jumping, physics setup, camera behavior, and basic level layout, has been fully completed. Layer 2, which focuses on tilemap-based level creation, multi-scene setup, and the use of prefabs, is currently in progress. Additional level construction, hazards, and scene transitions will be completed as the team continues through the assigned tutorial sections.

Discussion

1. Explain what has proved to be harder (or easier) than expected. [10pts]

Several aspects of the project have proven more challenging than anticipated. Unity's current interface, 6000.3.6f1, has changed significantly since the Udemmy tutorial was recorded, 2019.3.0f1, which required frequent troubleshooting and adaptation. For example, the "Used By Composite" checkbox referenced in the tutorial no longer exists, and the workflow now relies on a Composite Operation dropdown. Unity's physics system also introduced unexpected issues, such as infinite jumping caused by missing grounded checks and collider overlap. Debugging these problems required careful testing and iteration. Additionally, the camera system initially failed to render the level in Play Mode due to an incorrect Z-position, resulting in a blue screen that required investigation to resolve. Version control also posed challenges, as the GitHub repository was initially linked to the wrong folder, necessitating a full migration of the `.git` directory into the correct project folder.

On the other hand, some tasks were easier than expected. Tilemap painting and level layout were intuitive once the Tile Palette was configured. Parallax backgrounds were straightforward to implement after linking the appropriate transforms in the camera script. Prefab creation and reuse across scenes worked smoothly once references were properly assigned.

2. What design revisions have you made to your game, based on what you've learned (in initial implementation)? [10pts]

Several design revisions were made based on early implementation feedback. Sprite slicing and Pixels Per Unit were adjusted to ensure crisp pixel art. Asset folders were reorganized to separate custom art from tutorial assets. Collider sizes and ground check positions were refined to improve platforming accuracy. The camera script was updated to include vertical clamping for better framing. Finally, the entire project was moved into a dedicated GitHub repository to streamline version control and collaboration.

3. Discuss the implementation challenges you faced. Were there aspects that anticipated building, but were unable to implement? [10pts]

The team encountered several implementation challenges during this phase. The infinite jump bug required identifying missing grounded checks and adjusting collider placement. The camera rendering issue required diagnosing the cause of the blue screen and correcting the camera's Z-position. Unity's API changes, such as the deprecation warning for velocity, required careful handling to maintain consistent physics behavior. The Composite Collider workflow also differed from the tutorial, requiring adaptation to Unity's updated interface. Finally, the GitHub repository migration was necessary to ensure that the correct project folder was tracked, which involved moving the `.git` directory and re-establishing version control.

P4

Zaman, Sharmin:

P4 Bibliography

Summary:

In “A Framework for Analysis of 2D Platformer Levels,” Smith, Cha, and Whitehead present a structured method for understanding how 2D platformer levels are designed, focusing on the components, patterns, and rhythms that shape player experience. The authors argue that level design is inherently genre-specific and that platformers require their own analytical vocabulary because their challenges, pacing, and spatial structures differ significantly from those of other genres. As they note, “good level design is vital for creating enjoyable games,” and platformers in particular rely on carefully crafted spaces that guide the player through movement, timing, and challenge.

The paper begins by situating platformer design within broader game design literature, pointing out that most books discuss level design only in general terms and rarely address genre-specific needs. To fill this gap, the authors analyze well-known platformers such as Super Mario World, Sonic the Hedgehog, and Donkey Kong Country 2, highlighting how each game emphasizes different types of challenge, mastery, speed, or exploration, yet still shares common structural elements.

The core contribution of the paper is a two-part model for analyzing platformer levels. The first part identifies five fundamental component categories: platforms, obstacles, movement aids, collectible items, and triggers. Each category is defined by its role in shaping gameplay.

Platforms form the navigable surfaces; obstacles create danger; movement aids help the player traverse the environment; collectibles provide rewards; and triggers alter the state of the level. These components are flexible and often overlap; for example, item boxes in Super Mario World function both as collectibles and as platforms.

The second part of the model introduces a hierarchical structural representation that organizes levels into rhythm groups, cells, and portals. Rhythm groups are small, self-contained segments of challenge that have a clear beginning, middle, and end. They represent the “phrases” of gameplay, similar to musical rhythms, and often correspond to sequences of jumps, timed movements, or enemy encounters. The authors emphasize that rhythm groups are essential for understanding pacing, noting that “breaks in rhythm groups are safe places for the player to rest before continuing the level.” These rest points help prevent fatigue and maintain engagement.

Cells are larger regions of linear gameplay composed of multiple rhythm groups. They represent stretches of the level where the player follows a mostly fixed path. Portals connect cells and allow for non-linearity, such as branching paths, secret areas, or alternate routes. This structure is particularly useful for analyzing games like Sonic the Hedgehog, where multiple paths coexist, and players can transition between them depending on speed, skill, or exploration.

The authors demonstrate their framework by analyzing a section of Sonic the Hedgehog 2, breaking it into rhythm groups and identifying how portals create branching paths. Their analysis shows how rhythm, pacing, and spatial structure work together to create challenge and flow. By decomposing levels into components and structural units, designers can better understand how to craft engaging platformer experiences and reuse successful patterns across levels.

Overall, the paper provides a clear vocabulary and analytical method for studying 2D platformer levels. It bridges the gap between intuitive level design and formal analysis, offering a toolset that can help both students and developers think critically about how platformer levels are constructed. The framework is especially valuable for understanding pacing, challenge, and player experience, all of which are essential considerations for designing effective platformer levels.

Citation (ACM format):

Smith, Gillian, Mee Cha, and Jim Whitehead. "A Framework for Analysis of 2D Platformer Levels." Sandbox '08: Proceedings of the 2008 ACM SIGGRAPH Symposium on Video Games, 2008, pp. 75–82.

ADD YOUR P4 HERE

Eccles, Kaidon:

Summary of Article:

The article "*What Do Game Developers Expect from Development and Design Tools?*" by Kasurinen, Strandén, and Smolander explores how game development organizations use technical tools and what they expect from them in normal practice. The study focuses on understanding the role of development tools in an industry that combines traditional software engineering with creative elements such as storytelling, art, and sound design.

The researchers conducted a qualitative study using interviews with seven game development organizations ranging from small startups to larger companies. These organizations varied in size, experience, and platforms, allowing the researchers to compare different approaches to development tools and infrastructure. The data collection involved multiple interview rounds

with project managers, developers, designers, and upper management, providing a broad perspective on how tools are selected and used in practice.

One of the most important findings of the study is that most game development organizations are generally satisfied with their existing tools. This result contrasts with earlier research that suggested significant dissatisfaction with development tools in the gaming industry. Instead of major issues, most developers reported only minor problems such as occasional bugs or usability concerns. The study concludes that modern tools are largely sufficient for supporting game development, especially as many companies adopt third party solutions rather than building their own tools from scratch.

A key insight from the article is the importance of flexibility and adaptability in development tools. Game development is highly iterative, meaning ideas evolve constantly throughout the process. As a result, developers value tools that allow them to quickly modify features, test ideas, and experiment with different designs. Rapid prototyping is especially critical, as teams often create early versions of gameplay concepts to evaluate whether they are engaging or functional. The ability to prototype quickly can significantly influence tool selection decisions.

The study also highlights that third-party game engines are widely used and preferred over internally developed ones. Creating a game engine is complex and resource-intensive, often involving hundreds of thousands of lines of code. Many organizations found that adopting established engines improved productivity and allowed teams to focus on gameplay and design rather than low-level technical systems.

Another important finding is that tool selection is not purely based on technical requirements. Instead, factors such as developer preference, prior experience, compatibility with existing systems, and team flexibility all play a role. In smaller organizations, developers often have the freedom to choose their own tools, which reflects a less formal and more agile development environment. Additionally, cost was surprisingly not a major factor for most organizations, even startups, suggesting that functionality and usability are prioritized over price.

The article also discusses challenges in game development, such as managing complex assets, coordinating teams, and balancing creative and technical requirements. While tools help address

these challenges, the study suggests that organizational practices and communication methods are equally important. Smaller teams often rely on informal communication rather than structured project management tools, whereas larger organizations tend to adopt more formal systems.

In conclusion, this article provides valuable insight into the practical needs of game developers. It emphasizes that effective development tools must support rapid iteration, adaptability, and creative design processes. While technical infrastructure is important, the success of game development also depends heavily on team dynamics and workflow practices. This research is useful for both developers and tool designers, as it highlights the features that are most critical for supporting modern game development.

Citation:

Kasurinen, J., Strandén, J.-P., & Smolander, K. (2013). *What do game developers expect from development and design tools?* Proceedings of the 17th International Conference on Evaluation and Assessment in Software Engineering (EASE '13). <https://doi.org/10.1145/2460999.2461008>

Beau Brechtel

Summary Of Article:

The article “MDA: A Formal Approach to Game Design and Game Research” by Hunicke, LeBlanc, and Zubek presents a structured framework for understanding and designing games. The framework is known as MDA, which stands for Mechanics, Dynamics, and Aesthetics. The authors argue that game design can be better understood by breaking games into these three interconnected components, each representing a different perspective of how games function and how players experience them.

Mechanics refer to the rules, systems, and basic components of the game. These include things like movement systems, scoring rules, physics, and algorithms that define how the game operates. Mechanics are what developers directly create and control during development. For example, in a platformer game, mechanics would include jumping, gravity, and collision detection. Dynamics happen when players interact with the mechanics over time. These are not directly programmed but arise from gameplay. For example, mechanics like limited resources

can lead to dynamics such as strategic decision-making or competition between players. Dynamics represent the real-time behavior of the game during play and are influenced by the player's choices. Aesthetics are the emotional responses and experiences of the player. The authors describe several types of aesthetic experiences, such as challenge, fantasy, discovery, fellowship, and expression. These are the goals of game design, as they define how the player feels while playing the game. For example, a horror game aims to create fear and tension, while a puzzle game focuses on satisfaction and problem-solving. The MDA framework is especially useful because it provides a common language for designers, developers, and researchers. It allows teams to analyze games more systematically and understand why certain games succeed or fail. Additionally, it helps designers predict how changes in mechanics may affect player experience, making it easier to iterate and improve a game.

One of the key ideas in the paper is that designers and players experience games in opposite directions. Designers start with mechanics and hope to produce certain aesthetics, while players begin with the aesthetic experience and interpret the game through dynamics and mechanics. This difference highlights the importance of carefully designing mechanics to ensure they lead to a player experience that the designer intends.

Overall, this paper is significant because it bridges the gap between technical game development and player-centered design. It emphasizes that successful games are not just about code or graphics, but about creating meaningful and engaging player experiences. The MDA framework remains widely used in both academic research and industry practice, making it a foundational concept in modern game design.

Evans, Nathan:

Souki, Francisco. "The Design Direction of *I Expect You To Die 2*." *Game Developers*

Conference 2022, Schell Games, GDC Vault, 2022,

<https://gdcvault.com/play/1028050/The-Design-Direction-of-I>

Francisco Souki's article explains how Schell Games handled design direction for *I Expect You To Die 2*. The article focuses on the work behind a finished VR puzzle game, including design documents, production stages, team communication, and playtesting. Souki presents design direction as the shared plan that keeps a game focused while many people build different parts of it. This source is useful because it shows how a team protects the player experience before the final version reaches players.

One of the strongest ideas in the article is the value of design documents. Souki explains that the team used real development documents to guide production and keep decisions organized. These documents helped their team record puzzle goals, design rules, tone, and expectations for each stage of their project. This was extra important for them as this game is a sequel to *I Expect You To Die*. They had to keep the spy theme, escape room style, humor, and hands-on VR puzzles from the first game while still adding fresh scenes and new puzzle ideas.

The article also explains why design direction matters for team communication. Game development involves designers, artists, writers, programmers, and producers all working on different parts of the same player experience. Souki shows that a shared design direction gives everyone the same target. It helps the team make choices, cut features, and adjust the game during production without losing the main idea.

The article gives a strong reason for playtesting in puzzle design. In a VR puzzle game, players do things in different ways. Some players may figure out the right use for an item right away. Others may carry it around, try it in the wrong spot, or miss the clue completely. Because of that, these puzzle designers have to watch real players. Souki explains that playtesting helped both the development team and the public shape the direction of the game. This makes sense because a puzzle only works when players can understand the goal, test ideas, and get useful feedback from the scene.

This article connects well to our game, *Hamster Homebound*, even though our game was much smaller. This connects to our game too. Since *Hamster Homebound* is a 2D platformer, we had to think about how each part felt while playing. All of our work had to fit together so the levels made sense and felt playable. Overall, this source fits a

game design bibliography because it shows that strong games come from planning, testing, shared decisions, and a clear goal. The biggest lesson I took from the article is that design direction gives a team a way to judge ideas and keep the final game consistent.

Schroeder, Nicholas:

Zhang, C., Yu, S. & Ji, J. CFI: a VR motor rehabilitation serious game design framework integrating rehabilitation function and game design principles with an upper limb case. *J NeuroEngineering Rehabil* **21**, 113 (2024). <https://doi.org/10.1186/s12984-024-01373-2>

Zhang et. All in their paper describe the creation of a novel game design framework, customized specifically for VR technology and games to help with medical rehabilitation of motor function disorders. Patients with such motor impairments can gain back part of their lost movement, and by introducing this VR game design framework, it creates a more motivating environment than typical monotonous rehabilitation methods. The term “VR rehabilitation serious” used references their distinction between VR entertainment/commercial games utilized in recovery programs and VR games designed specifically with elements of recovery in mind, to maximize educational aspects (teaching medical rehabilitation) or purposeful recovery-oriented gameplay.

The paper suggests a lack of focus for game-design frameworks at the implementation of VR in medical rehabilitation, where most researchers have focused on elements of improving games by adding elements of VR rather than looking at VR game design methods as rehabilitation structures.

One area more explored in using Head-Mounted Displays (HMD) in VR rehabilitation therapy, but authors suggest a lack of sufficient quality for engagement, proper integration of VR

mechanisms not found in traditional methods, or effective feedback from gameplay. Similarly, game design frameworks like MDA (Mechanics, Dynamics, and Aesthetics) were improved over time, into frameworks like DDE (Design-Dynamic-Experience), DGE (Design-Game-Experience), and GFI (Goal-Feedback-Interpretation) that took steps to more clearly refine the process for viable medical application while not limiting creation to game design specialists.

The paper also compares and organizes definitions for important terms across reviewed literature, such as “difficulty” (“challenge”), “reward” (“achievement”) or “motivation”, “goal” (“purpose”), “choice”, and “immersion”. Ideally, these elements in games should be implemented in VR-optimized reward-cycle to best mimic clinical rehabilitation methods while adding more “interest” / “motivation” in the exercises from the patient over typical long rehabilitation periods. It also emphasizes in its specific model the importance of three aspects: Clinical – the side that pertains to rehab practice goals and objectives matching game rules and settings; Function – the side that embraces the functionality of VR serious games that connect the clinical result wanted with maintained motivation in the game; and Interesting – the side that holds patients closer to recovery by keeping them engaged with simple rewards, complex incentives, and designed cycles to build an intrinsic relationship with the game to keep them

Game Reflection

Section 3 is Game Reflection (expected from each person for your contribution): Reflections and Lessons Learned

Individual:

3.1. YourLastName What We Are Most Proud Of (Individually)

Zaman, Sharmin: setting up game environment, camera, health system UI, pickups, Milton animations

I am most proud of the character animation work I completed for Milton. I came into this project with no prior experience in animation, and while the tutorial made the process look simple, getting the animations to feel smooth and natural required a lot of trial, error, and frame-by-frame adjustments. Beyond the technical work, I'm proud of persisting through the project. Even when I made a mistake that caused me to accidentally delete a UI prefab, which forced me to rebuild the entire UI Canvas from scratch, I stuck with it and saw the project through to the end.

Eccles, Kaidon: I am proud of the level select scene and visual transitions. It was a bit tricky at times when scripting because there were many different scenes or actions that needed transitions and as such many scripts that needed editing to reference the correct processes. All of that work is to provide the player with smooth, seamless-appearing gameplay. It was tedious juggling the many referencing scripts but once it was functioning correctly it was mentally rewarding.

Evans, Nathan: I worked on several parts of the project, including the Level 3 tilemap, tutorial text, checkpoint guidance, frog enemy behavior, audio integration, UI adjustments, and Git/GitHub management. I also worked on tutorial messages for movement, jumping, double jumping, spikes, checkpoints, stars, and pickups so the player had more guidance while learning

the game. I am most proud of helping bring together gameplay pieces that made the game feel more complete, especially the Level 3 platforming section and the frog enemy interaction. The frog enemy took a lot of testing because it needed to damage the player on normal contact but also be defeated when the player double jumped on it correctly. I also setup the audio for the whole game, and I am pretty proud of that, since I had never done audio within game design before this.

Brechtel, Beau: I am proud of the launch/main menu screen I was able to create for the game. Although simple, the start screen is important because it is what the players first see when launching the game and can affect how they view the game and even if they want to continue to play the game itself.

Schroeder, Nicholas: I am not an artist at all, but I have wanted to try branching out to learn more skills adjacent to game design (like pixel art), and I am pretty content with how some of the sprite animations for Milton and the family came out. I watched quite a few Youtube tutorials and had a lot of trial and error to tweak it towards what I wanted, but I think it was well worth the experience despite not being game programming and came out well.

3.6. What We'd Do Differently. (Individually) Each person may have a different perspective on 3.1 & 3.6 <3.1.YourLastName> where individual contribution).

Zaman, Sharmin: If I were to approach this project again, I would choose a less detailed tutorial. The one we followed was extremely thorough and geared toward perfectionists who want to fine-tune every small detail. With the limited time we had, this level of depth made the project more ambitious than necessary. I believe this contributed to the project taking longer than expected, especially when combined with external factors. A more general, streamlined tutorial

would have allowed us to focus on core mechanics and polish without getting overwhelmed by complexity.

Eccles, Kaidon: I agree with Sharmin about the detailed tutorial. Especially with having a section that could only be fully developed once most of the game's functionality was fully implemented. At first it was slightly overwhelming and difficult to get an idea of exactly where everything was. However, just sitting down and taking a good look at the processes and my group's documentation of their sections assisted with that greatly. A more simple tutorial would have resulted in a better polished and longer game, but I am still happy with our results and have enjoyed working on this project.

Evans, Nathan: If I were to approach this project again, I would handle Git much more carefully. A lot of the issues I ran into were my fault because I did not always fully understand how Unity scene files, branches, and commits worked together, since this was my first time using Git. There were times when I should have checked Git more carefully before making changes or pushing my work. Next time, I would save my scene, pull updates, and check with the group before working on files that someone else might also be editing. Even though the Git problems were frustrating, they helped me learn how important version control is when working on a team project.

Brechtel, Beau: If I were to approach this project again I would try to figure out a more organized way for us all to work on the game at the same time. There were times where two people might be working on the same scene and then one of those person's changes would be totally overwritten as a result. There were also points in time where one person couldn't work on their part until someone else had finished their section. Overall, if I were to do this project again

I would want to come up with a more structured plan on who is working on what and when so that we aren't all trying to work on things towards the end of the semester.

Schroeder, Nicholas: If I had to do it over again, I would have taken more time to plan out the sprites and cross-reference all our art resources we used to make sure that my sprites would fit seamlessly into the art style of the game. It didn't fit badly, but the aspect ratio of the player to screen size is a lot higher than expected (the player ended up being quite large on screen), so some details that I thought wouldn't matter as much stood out to me as worse, and I could have improved them had I known about it ahead of time.

3.2. Changes to Original Game Design (Technical)

- Changed enemies from Snake and Big Snake to Frog and Hawk
- Opted not to implement red key
- Changed level design from planned blueprint

3.3. Learned from Playtesters

We learned a lot from the playtesters through our pre-questionnaire. We saw that we needed to make the objective of the game clearer. We saw that they liked the visuals and camera work.

Before sending out the post questionnaire, we focused on improving these areas. From the post questionnaire, we found that we still need to balance out the difficulty in levels because evidently, we may have accidentally made level 1 too complex.

3.4. What Changes would you make based on what you learned from Playtesting

Based on what we learned from playtesting, we would make the gap that Milton has to jump over in level 1 smaller and easier to cross, as we think this may have contributed to the difficulty of the level.

3.5. If We Had More Time

If we had additional development time, we would:

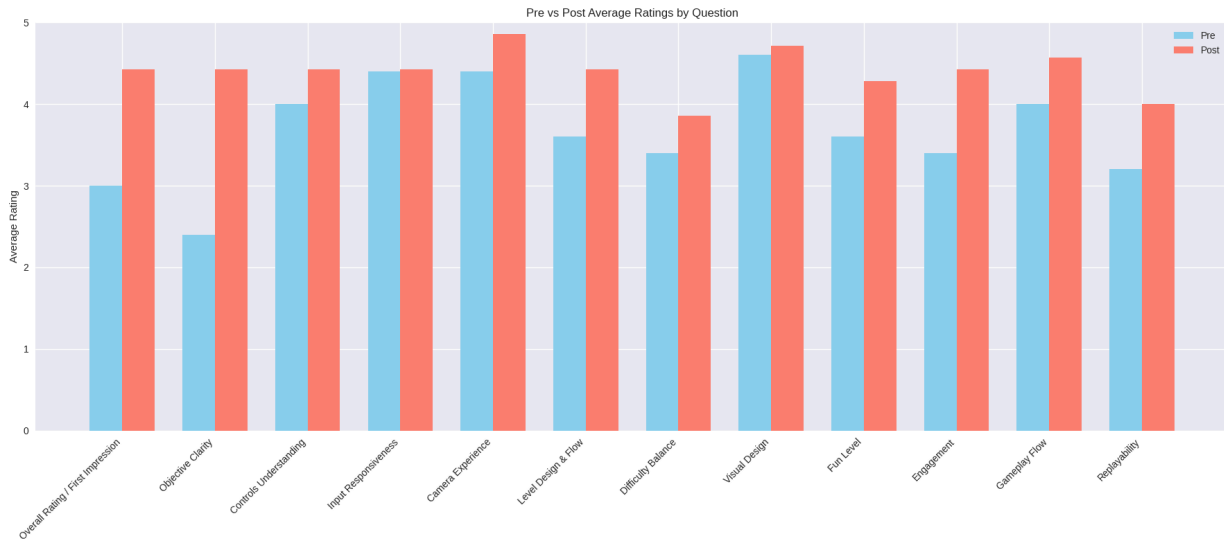
- Add proper level transitions
 - Create more levels
 - Replace the Level 1 speech bubble with a bottom-screen textbox
 - Simplify Level 1's layout
 - Add pause option
 - Add mute music option
 - Add Continue button to Main Menu
 - Add a moving platform to level 3
 - Polish enemy AI and animations
-

P5

Testing Results

Our testing method was a questionnaire. We took our testing rubric and converted it into two Google Forms, a pre and post questionnaire form so it would be easier to distribute and acquire results from. We had 5 playtesters test out the game in its intermediate phase and complete a pre-questionnaire and 5 playtesters test out the game in its last phase and complete the post questionnaire. Both forms had pretty much the exact same questions.

This bar graph compares the average of the pre-questionnaire answers against the average of the post-questionnaire answers by question:



In pre, we were rated highest on visual design and lowest on objective clarity. In post, we were rated highest on camera experience and lowest on difficulty balance. Based on the graph, we seem to have improved the game enough by the time of release of post to have received better ratings overall.

The questions that showed the most difference in terms of rating in pre vs post evaluation, were 1. How would you rate the game? and 2. How clear was the objective of the game?. These two improved vastly in post after we implemented our speech bubble in Level 1 which guided the user through the tutorial and finished the major functionalities of our game

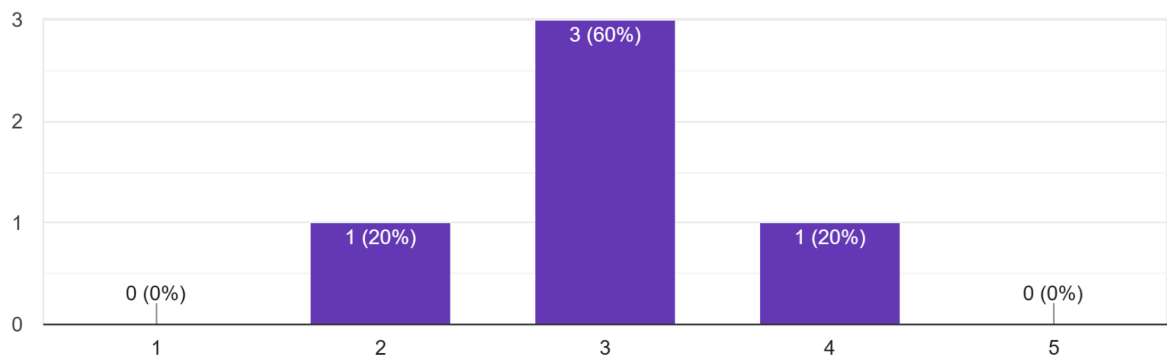
These were the responses to each:

1. How would you rate the game?

Pre:

How would you rate your first impression of the game?

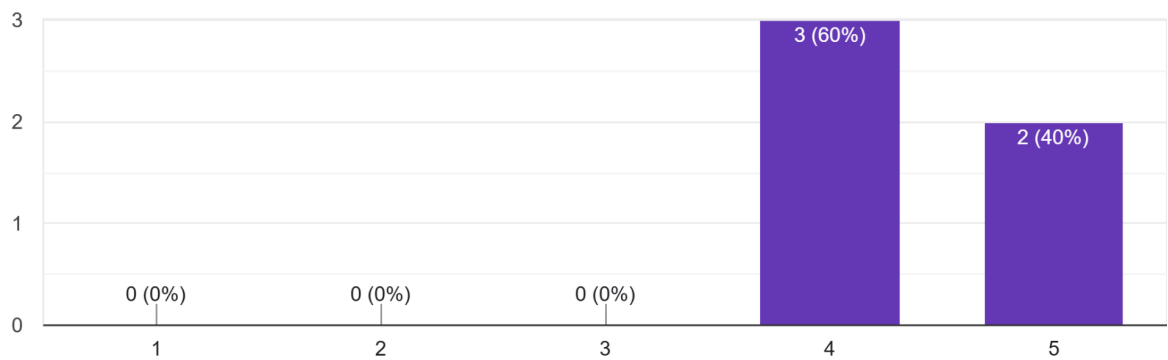
5 responses



Post:

How would you rate the game overall?

5 responses

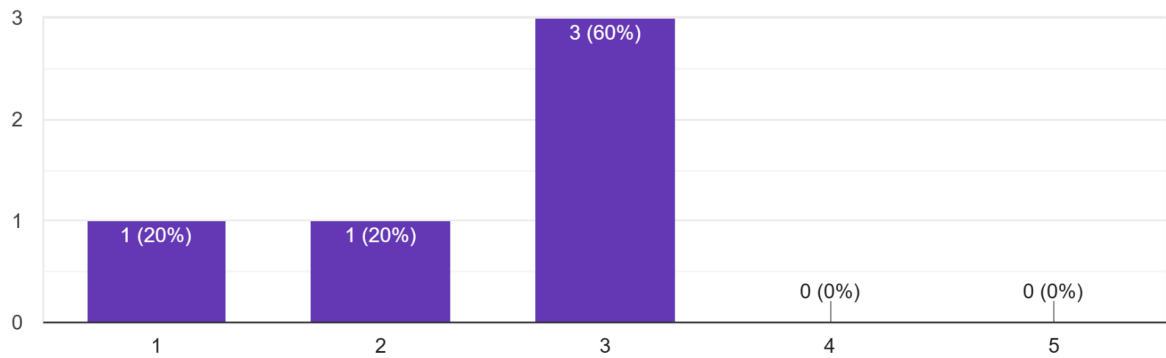


2. How clear was the objective of the game?

Pre:

How clear was the objective of the game?

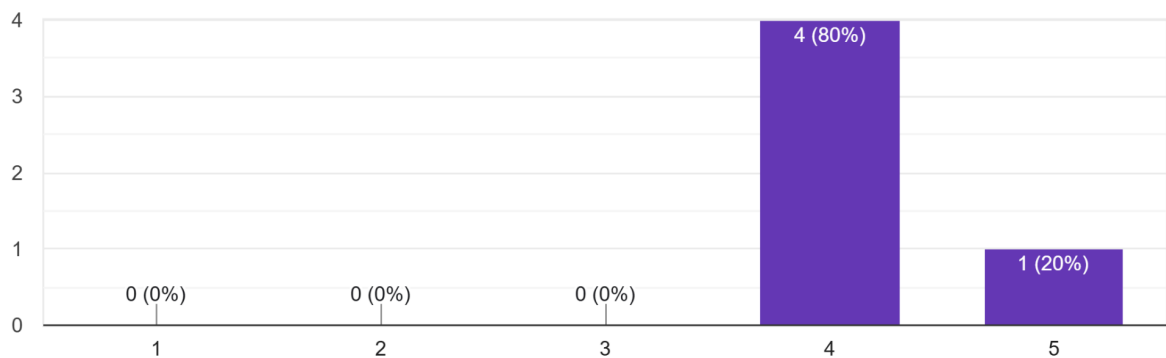
5 responses



Post:

How clear was the objective of the game?

5 responses



The area we have the most room for improvement on, based on the averages from post, is our difficulty balance.